

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

dataset = pd.read_csv('/content/Boston.csv')
dataset
```

	Unnamed: 0	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black	lstat	medv
0	1	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	2	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	3	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	4	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	5	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	502	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	503	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	504	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	505	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	506	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 15 columns

Next steps: [Generate code with dataset](#) [View recommended plots](#) [New interactive sheet](#)

```
dataset.shape
```

(506, 15)

```
dataset.head()
```

	Unnamed: 0	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black	lstat	medv
0	1	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	2	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	3	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	4	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	5	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2

Next steps: [Generate code with dataset](#) [View recommended plots](#) [New interactive sheet](#)

```
dataset.tail()
```

	Unnamed: 0	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black	lstat	medv
501	502	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	503	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	504	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	505	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	506	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Unnamed: 0  506 non-null    int64
1   crim        506 non-null    float64
```

```
2  zn          506 non-null  float64
3  indus       506 non-null  float64
4  chas        506 non-null  int64
5  nox         506 non-null  float64
6  rm          506 non-null  float64
7  age         506 non-null  float64
8  dis         506 non-null  float64
9  rad         506 non-null  int64
10 tax         506 non-null  int64
11 ptratio     506 non-null  float64
12 black       506 non-null  float64
13 lstat       506 non-null  float64
14 medv        506 non-null  float64
dtypes: float64(11), int64(4)
memory usage: 59.4 KB
```

```
dataset.describe()
```



	Unnamed: 0	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	
count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506
mean	253.500000	3.613524	11.363636	11.136779	0.069170	0.554695	6.284634	68.574901	3.795043	9.549407	408.237154	1.023319
std	146.213884	8.601545	23.322453	6.860353	0.253994	0.115878	0.702617	28.148861	2.105710	8.707259	168.537116	0.846451
min	1.000000	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000	2.900000	1.129600	1.000000	187.000000	1.000000
25%	127.250000	0.082045	0.000000	5.190000	0.000000	0.449000	5.885500	45.025000	2.100175	4.000000	279.000000	1.000000
50%	253.500000	0.256510	0.000000	9.690000	0.000000	0.538000	6.208500	77.500000	3.207450	5.000000	330.000000	1.000000
75%	379.750000	3.677083	12.500000	18.100000	0.000000	0.624000	6.623500	94.075000	5.188425	24.000000	666.000000	2.000000
max	506.000000	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000	100.000000	12.126500	24.000000	711.000000	2.000000

```
dataset.isnull().sum()
```



	0
Unnamed: 0	0
crim	0
zn	0
indus	0
chas	0
nox	0
rm	0
age	0
dis	0
rad	0
tax	0
ptratio	0
black	0
lstat	0
medv	0

dtype: int64

```
type(dataset)
```



```
pandas.core.frame.DataFrame
def __init__(data=None, index: Axes | None=None, columns: Axes | None=None, dtype: Dtype |
None=None, copy: bool | None=None) -> None

/usr/local/lib/python3.11/dist-packages/pandas/core/frame.py
Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Data structure also contains labeled axes (rows and columns).
Arithmetic operations align on both row and column labels. Can be
thought of as a dict-like container for Series objects. The primary
```

```
data = pd.DataFrame()
data.isnull().sum()
```



```
0
```

```
dtype: float64
```

```
import seaborn as sns
sns.distplot(dataset.medv)
```



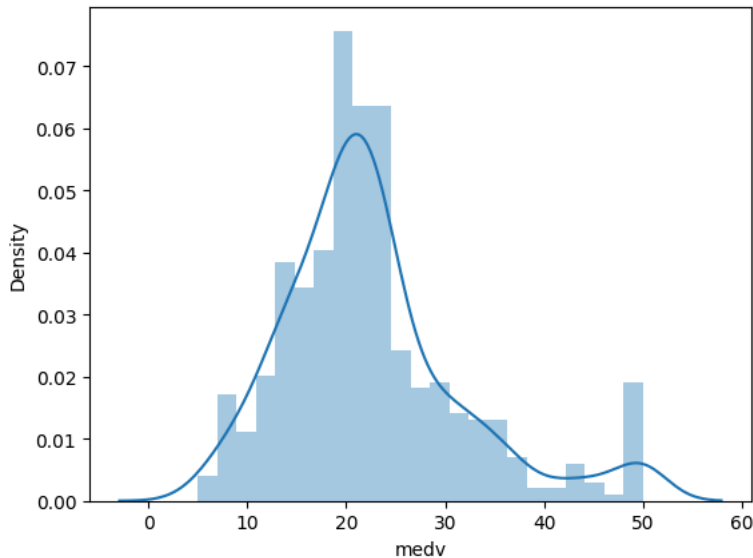
```
<ipython-input-18-bd938901a885>:2: UserWarning:
```

```
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.
```

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

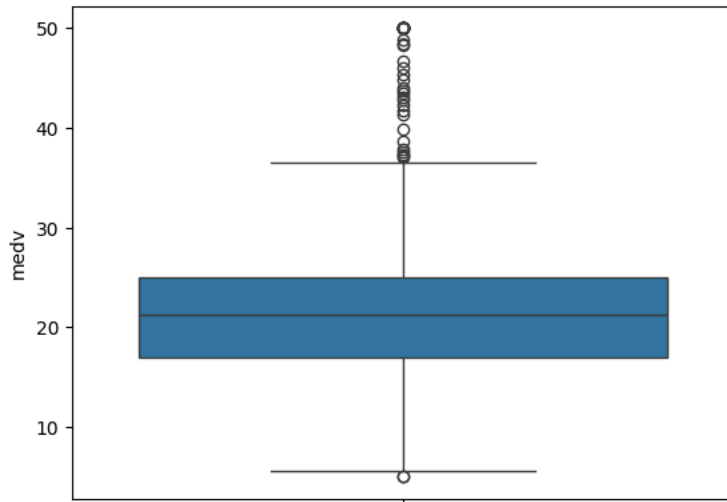
For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(dataset.medv)
<Axes: xlabel='medv', ylabel='Density'>
```



```
sns.boxplot(dataset.medv)
```

<Axes: ylabel='medv'>

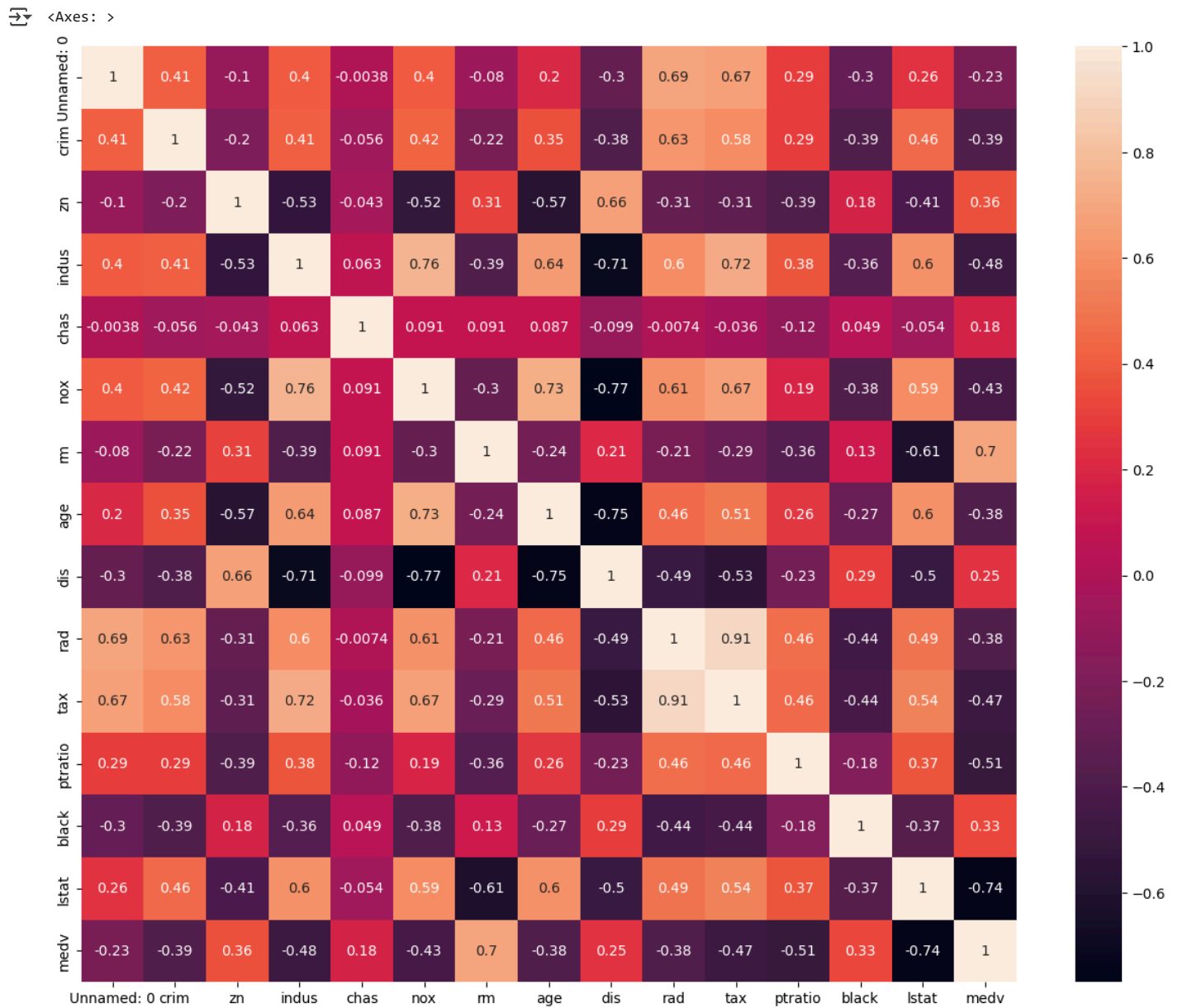


```
correlation = dataset.corr()
correlation.loc["medv"]
```

<Axes: ylabel='medv'>

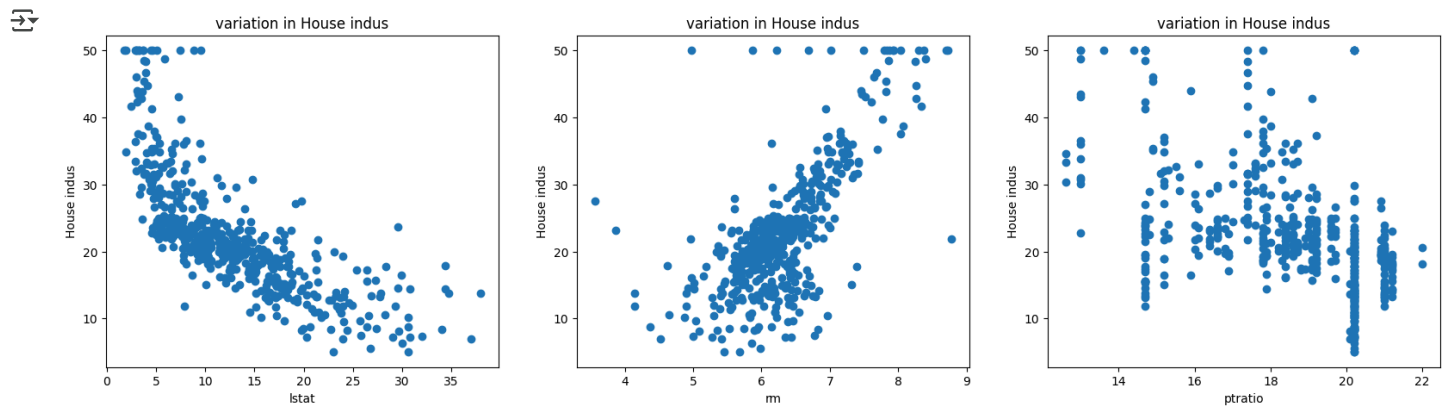
	medv
Unnamed: 0	-0.226604
crim	-0.388305
zn	0.360445
indus	-0.483725
chas	0.175260
nox	-0.427321
rm	0.695360
age	-0.376955
dis	0.249929
rad	-0.381626
tax	-0.468536
ptratio	-0.507787
black	0.333461
lstat	-0.737663
medv	1.000000

```
fig, axes = plt.subplots(figsize=(15, 12))
sns.heatmap(correlation, square = True, annot=True)
```



```
plt.figure(figsize=(20, 5))
features = ["lstat", "rm", "ptratio"]

for i, col in enumerate(features):
    plt.subplot(1, len(features), i+1)
    x = dataset[col]
    y = dataset.medv
    plt.scatter(x, y, marker="o")
    plt.title("variation in House indus")
    plt.xlabel(col)
    plt.ylabel("House indus")
```



```
x = dataset.iloc[:, :-1]
y = dataset.medv
dataset.columns

Index(['Unnamed: 0', 'crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis',
      'rad', 'tax', 'ptratio', 'black', 'lstat', 'medv'],
      dtype='object')
```

```
dataset.dtypes
```

```

0
Unnamed: 0    int64
crim         float64
zn           float64
indus        float64
chas         int64
nox          float64
rm           float64
age          float64
dis          float64
rad          int64
tax          int64
ptratio      float64
black        float64
lstat        float64
medv         float64
```

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)
print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```

(404, 14)
(102, 14)
(404,)
(102,)
```

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
x=dataset.iloc[:, :-1]
y=dataset.medv
```

```
mean = x_train.mean(axis=0)
std = x_train.std(axis=0)
x_train = (x_train - mean)/std
x_test = (x_test - mean)/std
```

```
from sklearn.linear_model import LinearRegression
```

```
regressor = LinearRegression()
```

```
sns.set(rc={"figure.figsize":(11.7,8.27)})
sns.distplot(dataset["medv"],bins=30)
plt.show()
```



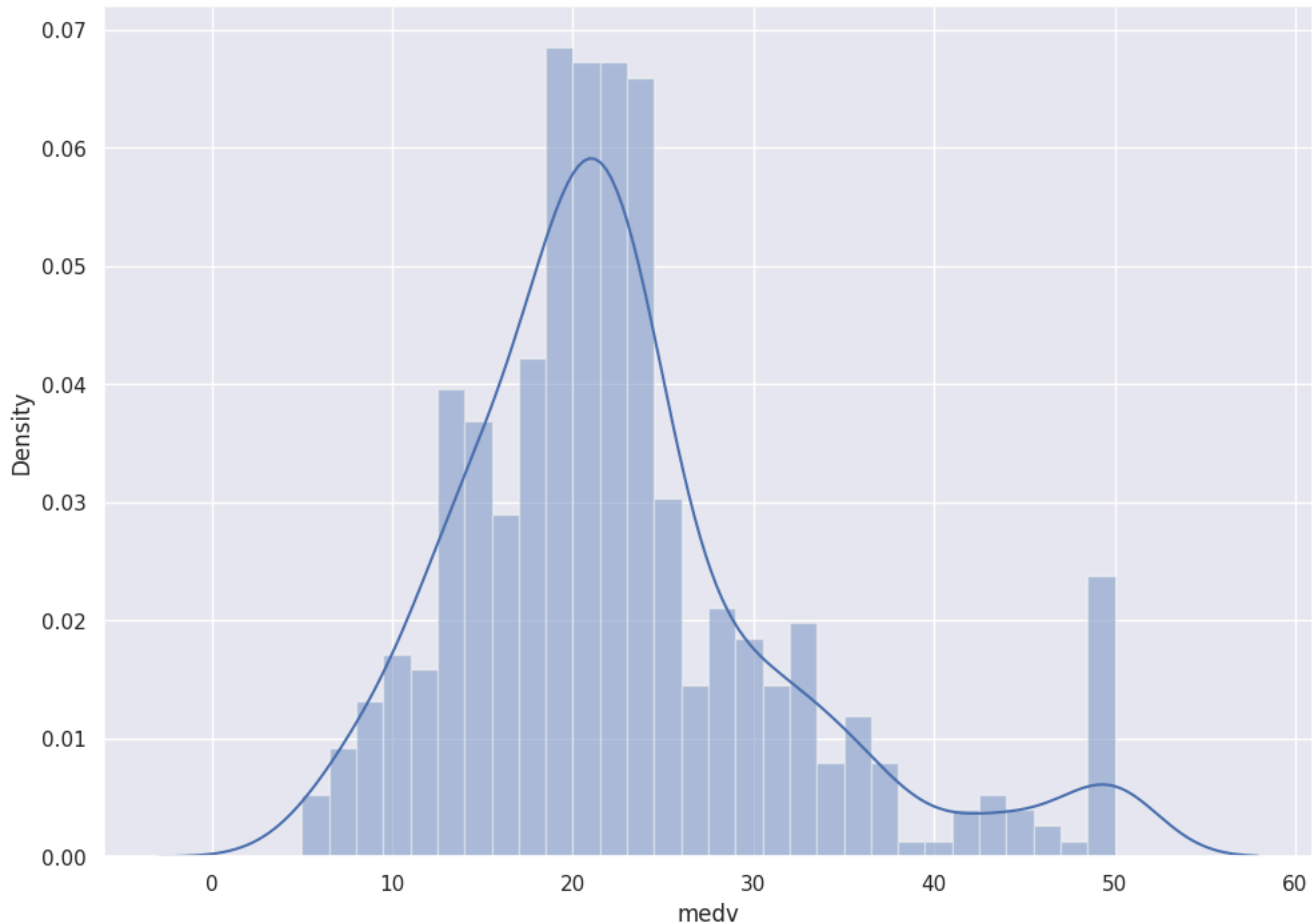
<ipython-input-25-c0bfdbe890e8>:17: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

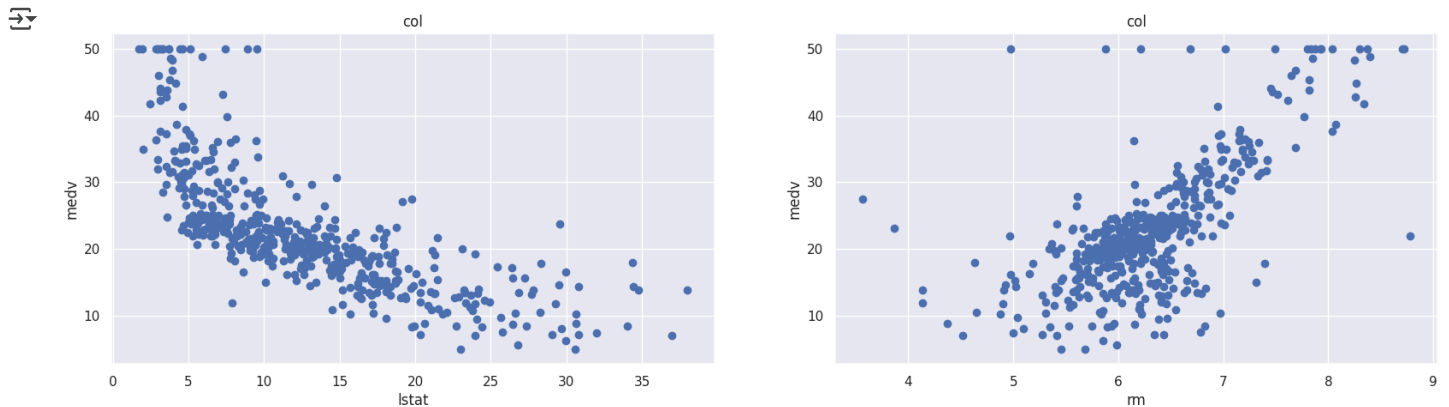
```
sns.distplot(dataset["medv"],bins=30)
```



```
x = pd.DataFrame(np.c_[dataset["lstat"], dataset["rm"]], columns=["lstat", "rm"])
y = dataset["medv"]
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)
print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(404, 2)
(102, 2)
(404,)
(102,)
```

```
plt.figure(figsize=(20, 5))
features = ["lstat", "rm"]
target = dataset["medv"]
for i,col in enumerate(features):
    plt.subplot(1,len(features),i+1)
    x = dataset[col]
    y = target
    plt.scatter(x,y,marker="o")
    plt.title("col")
    plt.xlabel(col)
    plt.ylabel("medv")
```



```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,r2_score
```

```
lin_model =LinearRegression()
lin_model.fit(x_train,y_train)
```

```
LinearRegression
```

```
y_train_predict =lin_model.predict(x_train)
rmse = (np.sqrt(mean_squared_error(y_train,y_train_predict)))
r2 = r2_score(y_train,y_train_predict)
print("The model performance for training ")
print(" ")
print("RMSE is {}",format(rmse))
print("R2 score is {}", format(r2))
print("\n")
```

```
The model performance for training
```

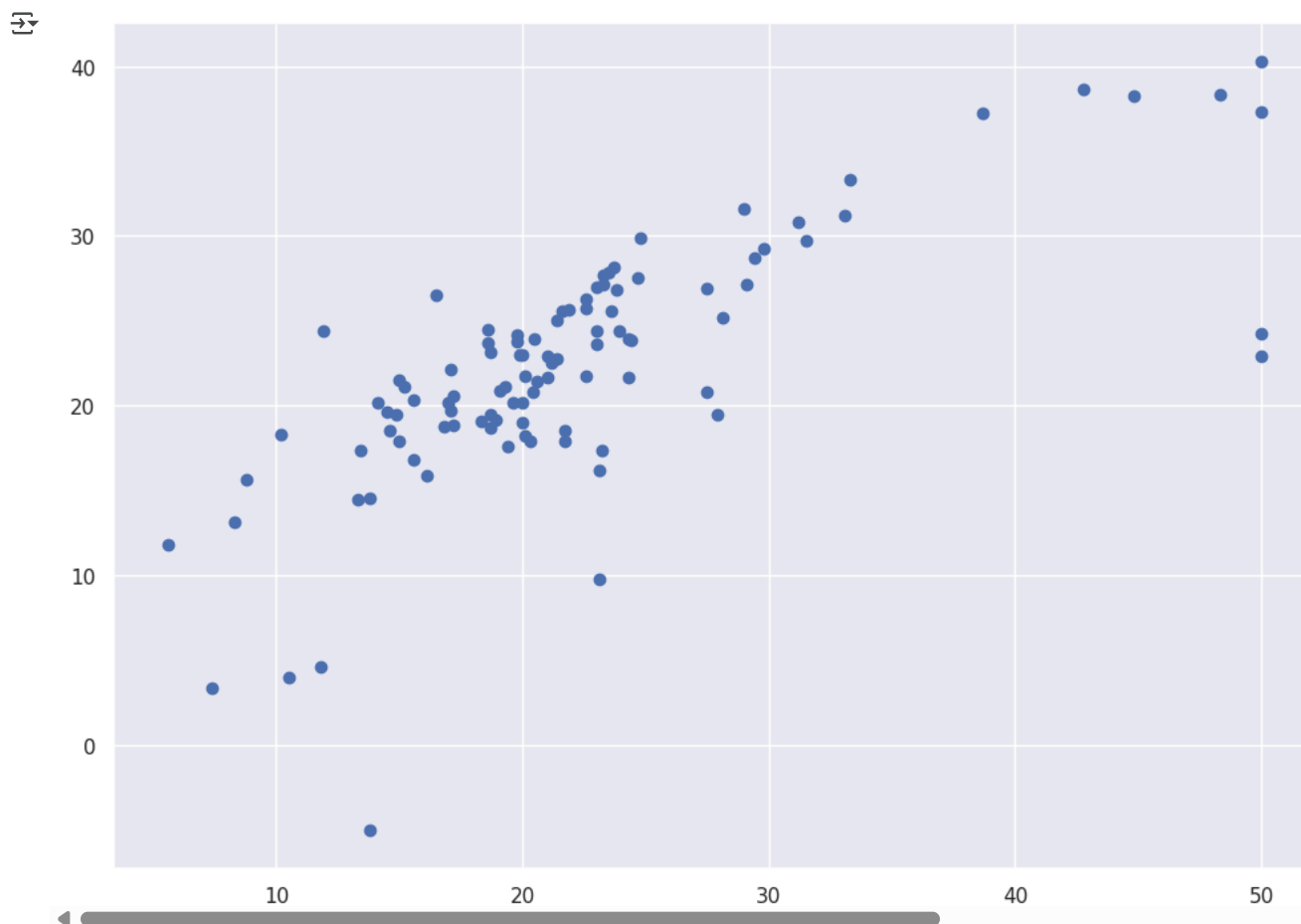
```
RMSE is {} 5.365657134224422
R2 score is {} 0.6618625964841893
```

```
y_test_predict =lin_model.predict(x_test)
rmse = (np.sqrt(mean_squared_error(y_test,y_test_predict)))
r2 = r2_score(y_test,y_test_predict)
print("The model performance for testing ")
print(" ")
print("RMSE is {}",format(rmse))
print("R2 score is {}", format(r2))
print("\n")
```


↗ The model performance for testing

RMSE is {} 6.114172522817781
R2 score is {} 0.5409084827186417

```
plt.scatter(y_test,y_test_predict)
plt.show()
```



```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)
```

```
import keras
from keras.layers import Dense, Activation, Dropout
from keras.models import Sequential
model = Sequential()
model.add(Dense(128, activation='relu', input_dim=13))
model.add(Dense(64, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(16, activation='relu'))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse', metrics=["mae"])
model.compile(optimizer='adam', loss='mean_squared_error', metrics=['mae'])
```

```
!pip install ann_visualizer
```

↗ Collecting ann_visualizer
 Downloading ann_visualizer-2.5.tar.gz (4.7 kB)
 Preparing metadata (setup.py) ... done
 Building wheels for collected packages: ann_visualizer
 Building wheel for ann_visualizer (setup.py) ... done
 Created wheel for ann_visualizer: filename=ann_visualizer-2.5-py3-none-any.whl size=4169 sha256=0185ac6e39095648d40848aeb03f054cd3d42c
 Stored in directory: /root/.cache/pip/wheels/28/4a/ad/e82da1aad2994e42bf0f4b1d403fdd8a64dfc38ae2c8a5daa4
 Successfully built ann_visualizer
 Installing collected packages: ann_visualizer
 Successfully installed ann_visualizer-2.5

```
!pip install graphviz
```

```
Requirement already satisfied: graphviz in /usr/local/lib/python3.11/dist-packages (0.20.3)
```

```
from ann_visualizer.visualize import ann_viz;
```

```
import keras
from keras.layers import Dense, Activation, Dropout
from keras.models import Sequential
model = Sequential()
model.add(Dense(128, activation='relu', input_shape=x_train[0].shape))
model.add(Dense(64, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(16, activation='relu'))
model.add(Dense(1))
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape` / `input_dim` argument to the `Dense` layer.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
x_train[0]
```

```
array([-0.41100022,  0.93567804])
```

```
k=4
num_epochs =100
num_samples=len(x_train)
all_scores=[]
model.compile(optimizer='adam', loss='mse', metrics=["mae"])
```

```
history = model.fit(x_train, y_train,epochs= num_epochs,verbose=1,batch_size=1, validation_data=(x_test,y_test))
```

```
Epoch 1/100
404/404 — 3s 3ms/step - loss: 379.7179 - mae: 14.9159 - val_loss: 34.0002 - val_mae: 3.7805
Epoch 2/100
404/404 — 2s 2ms/step - loss: 20.4201 - mae: 3.3609 - val_loss: 30.5430 - val_mae: 3.6409
Epoch 3/100
404/404 — 1s 2ms/step - loss: 21.6272 - mae: 3.5331 - val_loss: 30.0245 - val_mae: 3.2641
Epoch 4/100
404/404 — 2s 4ms/step - loss: 19.1871 - mae: 3.1517 - val_loss: 28.2761 - val_mae: 3.2749
Epoch 5/100
404/404 — 2s 2ms/step - loss: 20.2230 - mae: 3.3433 - val_loss: 27.6127 - val_mae: 3.1719
Epoch 6/100
404/404 — 1s 3ms/step - loss: 15.5757 - mae: 2.9634 - val_loss: 34.8920 - val_mae: 3.7118
Epoch 7/100
404/404 — 1s 3ms/step - loss: 16.7903 - mae: 3.0177 - val_loss: 28.5368 - val_mae: 3.3000
Epoch 8/100
404/404 — 1s 2ms/step - loss: 20.6225 - mae: 3.2599 - val_loss: 28.7151 - val_mae: 3.1934
Epoch 9/100
404/404 — 1s 2ms/step - loss: 19.1613 - mae: 2.9234 - val_loss: 29.5041 - val_mae: 3.2282
Epoch 10/100
404/404 — 1s 2ms/step - loss: 22.8152 - mae: 3.3486 - val_loss: 28.4630 - val_mae: 3.1598
Epoch 11/100
404/404 — 1s 2ms/step - loss: 16.8750 - mae: 2.8914 - val_loss: 29.9681 - val_mae: 3.1763
Epoch 12/100
404/404 — 1s 2ms/step - loss: 16.4024 - mae: 2.8839 - val_loss: 30.6133 - val_mae: 3.3464
Epoch 13/100
404/404 — 2s 4ms/step - loss: 15.6910 - mae: 2.7953 - val_loss: 28.8552 - val_mae: 3.3811
Epoch 14/100
404/404 — 2s 2ms/step - loss: 17.0655 - mae: 2.9214 - val_loss: 30.3065 - val_mae: 3.4273
Epoch 15/100
404/404 — 1s 2ms/step - loss: 17.2287 - mae: 2.9927 - val_loss: 28.8240 - val_mae: 3.3073
Epoch 16/100
404/404 — 1s 3ms/step - loss: 15.9161 - mae: 2.9477 - val_loss: 28.4268 - val_mae: 3.1633
Epoch 17/100
404/404 — 1s 2ms/step - loss: 20.1893 - mae: 3.1818 - val_loss: 29.9573 - val_mae: 3.3866
Epoch 18/100
404/404 — 1s 3ms/step - loss: 22.9949 - mae: 3.1979 - val_loss: 29.5228 - val_mae: 3.1900
Epoch 19/100
404/404 — 1s 2ms/step - loss: 15.6305 - mae: 2.9655 - val_loss: 34.1274 - val_mae: 4.1570
Epoch 20/100
404/404 — 1s 2ms/step - loss: 19.1113 - mae: 3.1108 - val_loss: 29.9298 - val_mae: 3.4429
Epoch 21/100
404/404 — 1s 2ms/step - loss: 17.9804 - mae: 3.0373 - val_loss: 29.0582 - val_mae: 3.2215
Epoch 22/100
```

```

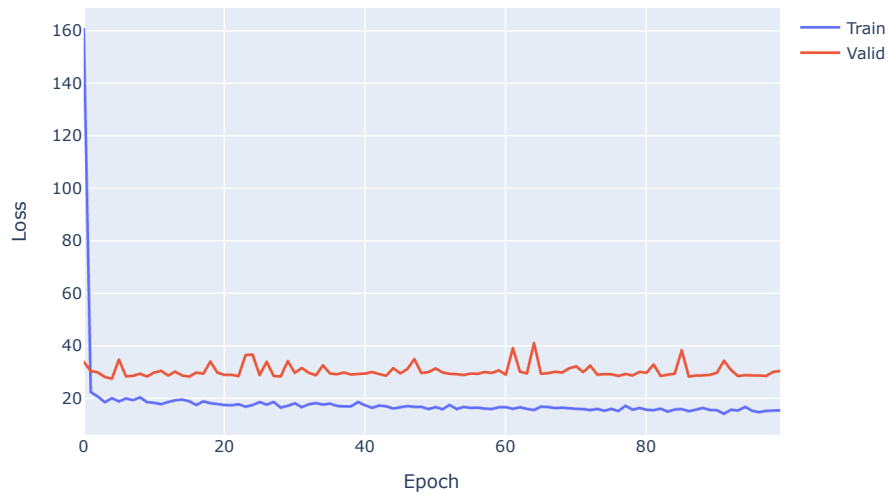
404/404 ————— 2s 4ms/step - loss: 24.9508 - mae: 3.3003 - val_loss: 29.1303 - val_mae: 3.1794
Epoch 23/100
404/404 ————— 2s 2ms/step - loss: 12.7504 - mae: 2.6981 - val_loss: 28.5867 - val_mae: 3.3244
Epoch 24/100
404/404 ————— 1s 2ms/step - loss: 13.7669 - mae: 2.7575 - val_loss: 36.6065 - val_mae: 4.1598
Epoch 25/100
404/404 ————— 1s 2ms/step - loss: 17.6930 - mae: 3.0918 - val_loss: 36.7889 - val_mae: 3.7851
Epoch 26/100
404/404 ————— 1s 2ms/step - loss: 17.2681 - mae: 2.9827 - val_loss: 28.9849 - val_mae: 3.2162
Epoch 27/100
404/404 ————— 1s 3ms/step - loss: 15.2544 - mae: 2.7635 - val_loss: 34.0863 - val_mae: 3.5673
Epoch 28/100
404/404 ————— 1s 2ms/step - loss: 21.1694 - mae: 3.1931 - val_loss: 28.6398 - val_mae: 3.2280
Epoch 29/100
404/404 ————— 1s 2ms/step - loss: 23.1955 - mae: 3.2644 - val_loss: 28.5124 - val_mae: 3.1448

```

```

from plotly.subplots import make_subplots
import plotly.graph_objects as go
fig=go.Figure()
fig.add_trace(go.Scattergl(y=history.history['loss'],name='Train'))
fig.add_trace(go.Scattergl(y=history.history['val_loss'],name='Valid'))
fig.update_layout(height = 500, width=700, xaxis_title="Epoch",yaxis_title="Loss")
fig.show()

```



```

fig=go.Figure()
fig.add_trace(go.Scattergl(y=history.history['mae'],name='Train'))
fig.add_trace(go.Scattergl(y=history.history['val_mae'],name='Valid'))
fig.update_layout(height = 500, width=700, xaxis_title="Epoch",yaxis_title="Mean Absolute Error")
fig.show()

```

